Lecture11: CRUD and Event in DOM

Once you have selected an element and stored it in a variable, you have a powerful object that gives you full control over that part of your webpage. Let's cover the three main categories of manipulation:

Editing What's Inside (Content & HTML)
Editing the Tag Itself (Attributes, Classes, and Styles)
Editing the Page Structure (Creating, Adding, and Removing Elements)

Part 1: Editing the Content Inside an Element

This is about what the user sees *inside* an element's opening and closing tags.

HTML Snippet for this section:

```
<div id="welcome-box">
  Please log in to see your messages.
</div>
```

```
// We've selected our target element
const welcomeBox = document.getElementById('welcome-box');
```

A. `.textContent` (For Plain Text - The Safe Default)

What it does: Gets or sets the **pure text** content of an element. It ignores and strips out all HTML tags.
Use Case: This should be your **default choice** for changing text on the page.

```
// Reading the text content
console.log(welcomeBox.textContent); // "Please log in to see your messages."

// Writing new text content
welcomeBox.textContent = "Welcome back, Alice!";
```

The Security Benefit: When you set `.textContent`, the browser treats your string as plain text. This is **safe** because it prevents malicious code from being executed.

```
// Even if this string came from a hacker...
const maliciousInput = "<script>alert('hacked!');</script>";

welcomeBox.textContent = maliciousInput;
// The browser will LITERALLY display the text: "<script>alert('hacked!');</script>"
// It will NOT run the script.
```

B. `.innerHTML` (For HTML - Powerful but Dangerous)

What it does: Gets or sets the **full HTML markup** inside an element.
Use Case: Use this **only when you need to generate HTML** and the content is from a **trusted source** (i.e., you, the developer).

```
// Let's create a more complex message
const welcomeMessage = `
  <h2>Welcome back, Alice!</h2>
  <p>You have <strong>5</strong> new messages.</p>
`;

// Writing new HTML content
welcomeBox.innerHTML = welcomeMessage;
```

Generated with PDFtion

The Security Risk (XSS - Cross-Site Scripting): Never use `.innerHTML` with content provided by a user (like a comment or a username). If a user enters malicious `<script>` code, `.innerHTML` will execute it, allowing the user to attack your website and its visitors.

Part 2: Editing the Element's Tag Itself

This is about changing the attributes of the tag, like its `id`, `class`, or `style`.

HTML Snippet for this section:

```
<div id="profile-pic-container" class="card">
  <img src="default-avatar.png" alt="User avatar">
</div>
```

```
const profileContainer = document.getElementById('profile-pic-container');
const profileImage = profileContainer.querySelector('img');
```

A. Changing Common Attributes (Direct Properties)

For standard attributes, you can often change them directly as properties on the element object.

```
// Change the image source
profileImage.src = 'images/alice.png';

// Change the alt text for accessibility
profileImage.alt = 'A photo of Alice';

// Change the ID of the container
profileContainer.id = 'user-123-avatar';
```

B. Changing CSS Classes (The `.classList` Toolbox)

This is the **best and safest way** to manage an element's classes. Forget about `.className`.

`.add('className')`: Adds a new class.
`.remove('className')`: Removes a class.
`.toggle('className')`: Adds the class if it's missing, removes it if it's there.
`.contains('className')`: Checks if an element has a class (returns `true` or `false`).

```
// Let's highlight the profile picture when the user is active
profileContainer.classList.add('is-active'); // Adds the 'is-active' class

// Let's remove the generic 'card' class
profileContainer.classList.remove('card');

// We can toggle a 'selected' state
profileContainer.classList.toggle('selected'); // Adds 'selected'
profileContainer.classList.toggle('selected'); // Removes 'selected'
```

Best Practice: Define your styles in a CSS file and use JavaScript's `.classList` to apply or remove those styles. This keeps your presentation (CSS) and logic (JS) separate.

C. Changing Inline Styles (The `.style` Property)

This allows you to directly apply CSS styles to an element's `style` attribute.

CRITICAL RULE: CSS properties with a hyphen are converted to **camelCase**.
`background-color` becomes `backgroundColor`.
`font-size` becomes `fontSize`.

```
const profileContainer = document.getElementById('profile-pic-container');

profileContainer.style.border = '2px solid blue';
profileContainer.style.borderRadius = '50%'; // Note the camelCase
```

Generated with PDFtion

```
profileContainer.style.width = '150px';      // Values must be strings with units
profileContainer.style.height = '150px';
```

Use Case: Best for dynamic styles that are calculated at runtime (e.g., setting an element's position based on the mouse). For static styles, prefer using `.classList`.

Part 3: Editing the Page Structure

This is the most powerful form of DOM manipulation.

A. Creating a New Element: `document.createElement()`

This creates a new element node in memory. It is **not yet on the page.**

```
// Create a new paragraph, but it's just floating in memory.
const newParagraph = document.createElement('p');

// Now, we configure it.
newParagraph.textContent = "This is a brand new paragraph created by JavaScript.";
newParagraph.classList.add('info-text');
```

B. Adding the New Element to the Page

HTML:

```
<div id="parent-container">
  <p id="first-child">First Paragraph</p>
  <p id="reference-element">I am the reference point.</p>
  <p id="last-child">Last Paragraph</p>
</div>
```

JavaScript:

```
// This is the new element we want to add.
const newElement = document.createElement('h2');
newElement.textContent = "I AM THE NEW ELEMENT";

// This is our main reference point for the examples.
const referenceElement = document.getElementById('reference-element');
```

Here are all the different ways you can add `newElement` to the page.

---

1. Adding Inside a Parent (as a Child)

These methods place the new element *inside* the boundaries of another element.

`.append(...nodes)` (Modern & Recommended)

What it does: Inserts the node as the **very last child** of the parent.
Use Case: The default way to add an item to the end of a list or container.

```
const parent = document.getElementById('parent-container');
parent.append(newElement);
```

Resulting HTML:

```
<div id="parent-container">
  <p id="first-child">...</p>
  <p id="reference-element">...</p>
  <p id="last-child">...</p>
  <h2>I AM THE NEW ELEMENT</h2>  <!-- Appended here -->
</div>
```

Generated with PDFtion

`.prepend(...nodes)` (Modern & Recommended)

What it does: Inserts the node as the **very first child** of the parent.
Use Case: Adding a new item to the top of a feed or a list.

```
const parent = document.getElementById('parent-container');
parent.prepend(newElement);
```

Resulting HTML:

```
<div id="parent-container">
  <h2>I AM THE NEW ELEMENT</h2>  <!-- Prepended here -->
  <p id="first-child">...</p>
  <p id="reference-element">...</p>
  <p id="last-child">...</p>
</div>
```

`.appendChild(node)` (Classic)

What it does: Same as `append`, but it's the older syntax. It can only add one node at a time and returns the appended node.

```
const parent = document.getElementById('parent-container');
parent.appendChild(newElement); // Same result as append
```

`.insertBefore(newNode, referenceNode)` (Classic & Powerful)

What it does: Inserts `newNode` into the `parentElement` **right before** the `referenceNode`. This is the classic way to insert into the middle of a list of children.
Use Case: Inserting an element at a specific position in a list.

```
const parent = document.getElementById('parent-container');
const referenceNode = document.getElementById('reference-element');
parent.insertBefore(newElement, referenceNode);
```

Resulting HTML:

```
<div id="parent-container">
  <p id="first-child">...</p>
  <h2>I AM THE NEW ELEMENT</h2>  <!-- Inserted before the reference -->
  <p id="reference-element">...</p>
  <p id="last-child">...</p>
</div>
```

---

2. Adding Next to an Element (as a Sibling)

These methods place the new element *outside* the boundaries of the reference element, at the same level in the DOM tree.

`.after(...nodes)` (Modern & Recommended)

What it does: Inserts the node **immediately after** the reference element, as its next sibling.
Use Case: Adding a new element directly following another one.

```
const referenceElement = document.getElementById('reference-element');
referenceElement.after(newElement);
```

Resulting HTML:

```
<div id="parent-container">
  <p id="first-child">...</p>
  <p id="reference-element">...</p>
  <h2>I AM THE NEW ELEMENT</h2>  <!-- Inserted here, as a sibling -->
```

Generated with PDFtion

```
  <p id="last-child">...</p>
</div>
```

`.before(...nodes)` (Modern & Recommended)

What it does: Inserts the node **immediately before** the reference element, as its previous sibling.
Use Case: Adding a new element directly preceding another one.

```
const referenceElement = document.getElementById('reference-element');
referenceElement.before(newElement);
```

Resulting HTML:

```
<div id="parent-container">
  <p id="first-child">...</p>
  <h2>I AM THE NEW ELEMENT</h2>  <!-- Inserted here, as a sibling -->
  <p id="reference-element">...</p>
  <p id="last-child">...</p>
</div>
```

`.insertAdjacentElement(position, element)` (The Most Versatile)

What it does: This is a highly flexible method that can do the job of all the others. You specify a `position` relative to the target element and the `element` to insert.
The Four Positions:

- `'beforebegin'`: Inserts the new element as a sibling, right before the target element (same as `.before()`).
- `'afterbegin'`: Inserts the new element as the first child of the target element (same as `.prepend()`).
- `'beforeend'`: Inserts the new element as the last child of the target element (same as `.append()`).
- `'afterend'`: Inserts the new element as a sibling, right after the target element (same as `.after()`).

Example (replicating `.after()`):

```
const referenceElement = document.getElementById('reference-element');
referenceElement.insertAdjacentElement('afterend', newElement);
```

Example (replicating `.prepend()`):

```
const parent = document.getElementById('parent-container');
// Note: we are calling this on the parent to insert AS A CHILD
parent.insertAdjacentElement('afterbegin', newElement);
```

There is also a related method, `.insertAdjacentHTML()`, which works like `.innerHTML` for these specific positions, but it carries the same security risks.

Summary Table: Which Method to Use?

Goal	Best Modern Method	Classic Method
Add as the LAST child	`parent.append(el)`	`parent.appendChild(el)`
Add as the FIRST child	`parent.prepend(el)`	`parent.insertBefore(el, parent.firstChild)`
Add BEFORE	`referenceEl.before(el)`	`parent.insertBefore(el, referenceEl)`

Generated with PDFtion

another element		
Add AFTER another element	`referenceEl.after(el)`	`parent.insertBefore(el, referenceEl.nextElementSibling)`

Recommendation: Stick to the modern methods (`append` , `prepend` , `before` , `after`) whenever possible. They are more intuitive, more flexible (can handle multiple nodes and text), and cover almost every use case you will encounter. Use `insertBefore` when you need to support older browsers or have a specific reference node to insert before.

C. Removing an Element

`.remove()` : The simplest and most modern way. You call it directly on the element you want to remove.

```
// Let's say we want to remove the profile image.
const profileImage = document.querySelector('#user-123-avatar img');

// First, check if the element was actually found before trying to remove it.
if (profileImage) {
  profileImage.remove(); // The image is now gone from the page.
}
```

This comprehensive set of tools (`.textContent` , `.classList` , `.style` , `createElement` , `append` , `remove`) gives you everything you need to transform a static HTML page into a fully dynamic application.

Optiimization

The First Thought:

> *"Every time I touch the live webpage, it's expensive. I should do all my work 'offline' on a temporary, invisible scratchpad, and then attach the finished result to the live page in one single, efficient step."*

That "invisible scratchpad" is a `DocumentFragment` .

The Problem: The High Cost of Touching the "Live" DOM

The DOM that is visible on the webpage is "live." Every time you change it—by appending a child, changing a style that affects size, etc.—you force the browser to do work.

DOM Update: The element is added to the tree.
Reflow/Layout: The browser may have to recalculate the position and size of many elements on the page.
Repaint: The browser has to repaint the pixels for the updated part of the screen.

Now, imagine you need to add **1,000** items to a list.

The Inefficient "Bad" Way:
This code touches the live DOM in a loop.

```
const list = document.getElementById('my-list'); // This is a "live" DOM element.

// START THE TIMER
console.time("Loop without fragment");

for (let i = 0; i < 1000; i++) {
  const newItem = document.createElement('li');
  newItem.textContent = `Item ${i + 1}`;
```

Generated with PDFtion

```
  // This is the expensive part. We are modifying the live DOM 1,000 times.
  // This can cause up to 1,000 separate reflows and repaints.
  list.appendChild(newItem);
}

// STOP THE TIMER
console.timeEnd("Loop without fragment");
```

If you run this, you might notice a brief stutter or freeze in the browser. You are forcing the browser to do a massive amount of repetitive work. Each `appendChild` is a separate transaction with the rendering engine.

The Solution: The `DocumentFragment`

A `DocumentFragment` is a **lightweight, in-memory DOM node that is not part of the main document tree.** It's a temporary, invisible container.

It has the same API as a regular element: you can `.appendChild()` or `.append()` to it.
The key difference: Modifying a `DocumentFragment` is **extremely cheap**. Since it's not part of the visible page, changing it **does not trigger any reflows or repaints.**

The Efficient "Good" Way:
This code does all the heavy lifting "offline" and then updates the live DOM only once.

```
const list = document.getElementById('my-list');

// START THE TIMER
console.time("Loop WITH fragment");

// 1. Create the invisible "scratchpad".
const fragment = document.createDocumentFragment();

// 2. Do all your work on the scratchpad.
//    This loop is now very fast because it's only changing things in memory,
//    not on the visible page.
for (let i = 0; i < 1000; i++) {
  const newItem = document.createElement('li');
  newItem.textContent = `Item ${i + 1}`;

  // We append to the fragment, not the live list.
  fragment.appendChild(newItem);
}

// 3. Now, attach the entire finished result to the live DOM in ONE single operation.
list.appendChild(fragment); // This causes only ONE reflow and repaint!

// STOP THE TIMER
console.timeEnd("Loop WITH fragment");
```

If you run both of these code blocks, you will see that the version using the `DocumentFragment` is significantly faster.

The "Magic" of Appending a Fragment

When you append a `DocumentFragment` to another node, a special thing happens:

The `DocumentFragment` itself is **not** added to the DOM tree.
Instead, all of the **children** of the fragment are moved into the target element.
The `DocumentFragment` is left empty.

This makes it a perfect, single-use container for batching DOM updates.

Summary

Generated with PDFtion

Feature	`DocumentFragment`
What is it?	**An in-memory, "offline" DOM container.**
Why use it?	**To batch multiple DOM updates (adds, moves) into a single operation.**
The Benefit	**Dramatically improves performance by minimizing the number of expensive reflows and repaints.**
How to use it?	**1. `document.createDocumentFragment()` . 2. Append all your new elements to the fragment in a loop. 3. Append the single fragment to the live DOM element.**
When should I use it?	**Always use a `DocumentFragment` when you need to add a large number of elements to the DOM in a loop.**

Evenet Listener

Imagine your HTML page is a house. It has rooms (`div` s), furniture (`p` aragraphs), and interactive objects like a doorbell (`button`).

By itself, the doorbell is just a button on the wall. Pushing it does nothing. It's a static object.

An **event** is the **"ding-dong"** signal that the doorbell sends when it's pushed.
An **event listener** is the person **inside the house whose job is to listen for the doorbell.**
An **event handler** is the **action that person takes** when they hear the "ding-dong" (e.g., "get up and open the door").

In summary: An event is a signal that something happened. Your JavaScript code's job is to *listen* for these signals and *react* to them.

Without events, a webpage is a silent, static house. With events, it becomes a responsive home that reacts to the user.

In-Depth Guide to JavaScript Events

1. The Three Core Components

To make a webpage interactive, you always need three things:

The Target Element: Which HTML element are we waiting for an action on? (e.g., the `button`).
The Event Type: What specific action are we listening for? (e.g., a `'click'`).
The Listener/Handler: What code should be executed when that action happens? (e.g., a function that changes some text).

2. The Modern Way: `addEventListener()`

This is the standard, most powerful, and recommended way to handle events. You call it on the target element.

Syntax: `targetElement.addEventListener('eventType', functionToRun);`

Let's build a complete, simple example:

HTML:

```
<button id="action-button">Click Me</button>
<p id="status-text">Waiting for action...</p>
```

Generated with PDFtion

JavaScript:

```
// Step 1: Select the Target Element
const myButton = document.getElementById('action-button');
const statusText = document.getElementById('status-text');

// Step 2: Define the Handler Function (the "action plan")
// This function will be executed when the event occurs.
function onButtonClick() {
  console.log("Button was clicked!");
  statusText.textContent = "Action Performed!";
  statusText.style.color = 'green';
}

// Step 3: Attach the Listener
// We tell the button: "Hey, start listening for a 'click' event.
// When you hear one, execute the `onButtonClick` function."
myButton.addEventListener('click', onButtonClick);
```

Critical Point: We pass the function name `onButtonClick` directly. We do **not** call it with `()`. We are giving `addEventListener` a reference to our function—the "recipe"—to be used later.

```
// Mouse Events
element.addEventListener('click', (e) => console.log('Clicked'));
element.addEventListener('dblclick', (e) => console.log('Double clicked'));
element.addEventListener('mousedown', (e) => console.log('Mouse down'));
element.addEventListener('mouseup', (e) => console.log('Mouse up'));
element.addEventListener('mousemove', (e) => console.log('Mouse moving'));
element.addEventListener('mouseenter', (e) => console.log('Mouse entered'));
element.addEventListener('mouseleave', (e) => console.log('Mouse left'));
```

```
// Keyboard Events
document.addEventListener('keydown', (e) => {
console.log('Key pressed:', e.key);
console.log('Key code:', e.code);
console.log('Ctrl pressed:', e.ctrlKey);
console.log('Shift pressed:', e.shiftKey);
console.log('Alt pressed:', e.altKey);
});
```

```
document.addEventListener('keyup', (e) => console.log('Key released'));
document.addEventListener('keypress', (e) => console.log('Key press')); // Deprecated
```

```
// Form Events
const input = document.querySelector('input');
const form = document.querySelector('form');
```

```
input.addEventListener('focus', () => console.log('Input focused'));
input.addEventListener('blur', () => console.log('Input blurred'));
input.addEventListener('input', (e) => console.log('Input value:', e.target.value));
input.addEventListener('change', (e) => console.log('Changed:', e.target.value));
```

```
form.addEventListener('submit', (e) => {
e.preventDefault(); // Prevent form submission
console.log('Form submitted');
});
```

```
// Window Events
window.addEventListener('load', () => console.log('Page fully loaded'));
window.addEventListener('DOMContentLoaded', () => console.log('DOM ready'));
window.addEventListener('resize', () => console.log('Window resized'));
window.addEventListener('scroll', () => console.log('Page scrolled'));
```

Generated with PDFtion

3. The `event` Object: The Information Packet

When an event occurs and your handler function is called, the browser automatically passes a special object as the **first argument** to your function. This is the `event` **object**.

First Thought: "The `event` object is an information packet that tells me everything about what just happened."

Let's modify our handler to accept this object:

```
function onButtonClick(event) {
  // Let's inspect the information packet
  console.log(event);
}

myButton.addEventListener('click', onButtonClick);
```

If you run this and click the button, your console will show a `PointerEvent` or `MouseEvent` object filled with useful information: the exact x/y coordinates of the click, whether the Shift key was held down, and much more.

Two of the most important properties of the `event` object are:

A. `event.target`
This property tells you the **specific element that triggered the event**. This is incredibly useful when you have one listener on a parent element watching for clicks on many children.

Example:

```
<ul id="item-list">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

```
const list = document.getElementById('item-list');

list.addEventListener('click', function(event) {
  // 'this' would be the <ul>, but 'event.target' is the specific <li> that was clicked!
  console.log("You clicked on:", event.target.textContent);
});
```

This pattern, called **event delegation**, is very efficient.

B. `event.preventDefault()`
This method stops the browser's **default behavior** for an element.

Classic Use Case: Form Submission
By default, when you submit an HTML form, the browser tries to send the data to a server and **reloads the page**. We usually want to stop this in a modern web app.

HTML:

```
<form id="my-form">
  <input type="text" id="username">
  <button type="submit">Submit</button>
</form>
```

JavaScript:

```
const myForm = document.getElementById('my-form');

myForm.addEventListener('submit', function(event) {
  // STOP the browser from reloading the page.
```

Generated with PDFtion

```
  event.preventDefault();

  const usernameInput = document.getElementById('username');
  console.log(`Form submitted with username: ${usernameInput.value}`);
  // Now we can send this data using fetch() without a page refresh.
});
```

---

4. Common Event Types to Know

Mouse Events:

- `click` : A single click.
- `dblclick` : A double click.
- `mousedown` : When the mouse button is pressed down.
- `mouseup` : When the mouse button is released.
- `mouseover` : When the mouse pointer enters an element.
- `mouseout` : When the mouse pointer leaves an element.
- `mousemove` : Fires continuously as the mouse moves over an element.

Keyboard Events:

- `keydown` : When a key is pressed down.
- `keyup` : When a key is released.
- `keypress` : (Older, avoid) Fires when a key that produces a character is pressed.

Form Events:

- `submit` : When a form is submitted.
- `input` : Fires immediately when the value of an `<input>` , `<select>` , or `<textarea>` changes.
- `change` : Fires when the value changes and the element loses focus.

Window Events:

- `load` : Fires when the entire page (including images, scripts, etc.) has finished loading.
- `DOMContentLoaded` : Fires when the HTML document has been fully parsed and the DOM tree is ready (this is often a better choice than `load` as it fires earlier).
- `scroll` : Fires when the user scrolls the document.

---

5. Removing Event Listeners

If you add an event listener, especially one that fires often (like `mousemove` or `scroll`), it's good practice to clean it up when it's no longer needed to prevent memory leaks. You do this with `.removeEventListener()` .

CRITICAL RULE: To remove a listener, you must have a **reference to the exact same function** that was used to add it. This means you cannot use an anonymous function.

```
// This works:
function handleMouseMove() { /* ... */ }
window.addEventListener('mousemove', handleMouseMove);
window.removeEventListener('mousemove', handleMouseMove); // Correct!

// This does NOT work:
window.addEventListener('mousemove', () => { /* ... */ });
window.removeEventListener('mousemove', () => { /* ... */ }); // WRONG! This is a different anonymous
```

Bubbling and Capturing

The Setup

HTML Structure:

```
<div id="grandparent">GRANDPARENT
  <div id="parent">PARENT
    <button id="child">CHILD (Click Me)</button>
```

Generated with PDFtion

```
  </div>
</div>
```

JavaScript Code:
We will attach **only one** event listener, and it will be on the `grandparent`.

```
const grandparent = document.getElementById('grandparent');

grandparent.addEventListener('click', (event) => {
  console.log("--- Event Listener on GRANDPARENT was triggered ---");

  // The element the listener is attached to
  console.log("event.currentTarget:", event.currentTarget.id);

  // The element where the click originated
  console.log("event.target:", event.target.id);
});
```

(Note: `event.currentTarget` is the technically precise property for "the element the listener is on." In simple cases, it's the same as `this`, but it's clearer and works with arrow functions.)

The Action: You click on the `child` button.

Here is the complete, detailed flow of what the browser does, in order.

Phase 1: The Capturing Phase (The "Trickle Down")

The event is created and starts its journey from the top of the DOM tree down towards the target (`child` button). At each level, the browser checks for any *capturing* event listeners (`useCapture: true`).

Event starts at `window`. Browser asks: "Is there a *capturing* `click` listener on `window`?"
Answer: No. Event continues down.
Event reaches `document`. Browser asks: "Is there a *capturing* `click` listener on `document`?"
Answer: No. Event continues down.
Event reaches `<html>`. Browser asks: "Is there a *capturing* `click` listener on `<html>`?"
Answer: No. Event continues down.
Event reaches `<body>`. Browser asks: "Is there a *capturing* `click` listener on `<body>`?"
Answer: No. Event continues down.
Event reaches `<div id="grandparent">`. Browser asks: "Is there a *capturing* `click` listener on `grandparent`?"
Answer: No. Our listener was registered for the bubbling phase (the default). Event continues down.
Event reaches `<div id="parent">`. Browser asks: "Is there a *capturing* `click` listener on `parent`?"
Answer: No. Event continues down.

The capturing phase is now complete. **Nothing has been logged to the console yet.**

Phase 2: The Target Phase

The event has now arrived at the element that was directly clicked.

Event is on `<button id="child">`.
The browser checks for any `click` listeners (both capturing and bubbling) attached directly to the `child` button.
Answer: No. We didn't attach any listeners to the child.

The target phase is complete. **Still, nothing has been logged to the console.**

Phase 3: The Bubbling Phase (The "Bubble Up")

Generated with PDFtion

The event now turns around and starts its journey back up the DOM tree from the target. At each level, the browser checks for any *bubbling* event listeners (the default).

Event leaves `child` and bubbles to its parent, `<div id="parent">`.
Browser asks: "Is there a *bubbling* `click` listener on `parent`?"
Answer: No. Event continues up.

Event bubbles to the next parent, `<div id="grandparent">`.
Browser asks: "Is there a *bubbling* `click` listener on `grandparent`?"
Answer: YES! We attached one.
Action: The browser executes our callback function. It passes the `event` object that was created way back at the beginning of the process.

Inside the Callback:

`event.currentTarget` is the element this listener is running on: the `grandparent` div.
`event.target` is the element where the event originated: the `child` button.

The following is now printed to the console:

```
--- Event Listener on GRANDPARENT was triggered ---
event.currentTarget: grandparent
event.target: child
```

Bubbling Continues:
After our listener finishes, the event continues bubbling up to `<body>`, `<html>`, `document`, and `window`.
The browser checks for listeners at each of these levels. Since there are none, nothing else happens.

Event Ends: The event has completed its journey.

Summary and Conclusion

This flow perfectly demonstrates **Event Delegation**.

We put a "net" (the event listener) on a high-level ancestor (`grandparent`).
The event (`click`) happened on a deeply nested descendant (`child`).
The **bubbling phase** is the mechanism that guarantees the event signal will travel up from the `child` and eventually be "caught" by our net on the `grandparent`.
Inside the listener, the `event.target` property gives us the crucial information we need to identify the **original source** of the event, allowing us to distinguish between clicks on different grandchildren.



Generated with PDFtion